Running nf-core/rnaseq on OCI Ampere A1 Slurm with Apptainer

Date: 2026-06-23

Purpose

This guide documents the working path for running `nf-core/rnaseq` on an OCI Slurm cluster built on Ampere A1 compute. The validated path uses:

- Nextflow `26.04.3`
- nf-core/rnaseq `3.14.0`
- Slurm executor
- Apptainer runtime
- A custom ARM64 Apptainer toolbox image
- Kallisto pseudoalignment
- Shared cluster storage under `/config`

The guide intentionally uses the Kallisto path and skips STAR alignment because the tested A1 workers had one OCPU and about 16 GB RAM each.

Cluster Assumptions

The examples assume:

- Login node: `wise-insect-login`
- Controller node: `wise-insect-controller`
- Compute partition: `compute`
- Shared path visible from login and compute nodes: `/config`
- User: `ubuntu`
- Two A1 compute nodes, each configured as one Slurm CPU

Check the cluster:

```
sinfo -Nel
sinfo -s
squeue -u ubuntu
```

Every node that will run Nextflow tasks must have Apptainer:

```
for n in wise-insect-login $(sinfo -N -h -o '%N' | sort -u); do
  echo "== $n =="
  ssh -o BatchMode=yes "$n" 'apptainer --version && uname -m'
done
```

Expected architecture for A1:

```
aarch64
```

Directory Layout

Use shared storage so every Slurm worker sees the same files:

```
mkdir -p /config/GSE55190/{fastq,reference,results,containers}
mkdir -p /config/apptainer-cache
mkdir -p /config/nextflow-home
```

Do not put the work directory or input FASTQs only under the login node home directory. Slurm workers must be able to read them.

Acquire the Test Data

The benchmark dataset is GEO accession `GSE55190`, backed by SRA study `SRP038635`. The study contains 24 paired-end RNA-seq runs from mouse liver samples.

Use ENA FASTQ downloads rather than `fastq-dump GSE55190`. The GEO accession alone is not the correct direct input to `fastq-dump`; the real downloadable run accessions are `SRR1173457` through `SRR1173480`.

Create the run-to-sample mapping:

```
cd /config/GSE55190

cat > runs_to_samples.tsv <<'TSV'
SRR1173457■chow_wt_rep1
SRR1173458■chow_wt_rep2
SRR1173459■chow_wt_rep3
SRR1173460■chow_j1c_rep1
SRR1173461■chow_j1c_rep2
SRR1173462■chow_j1c_rep3
SRR1173463■chow_j2c_rep1
SRR1173464■chow_j2c_rep2
SRR1173465■chow_j2c_rep3
SRR1173466■chow_alab_rep1
SRR1173467■chow_alab_rep2
SRR1173468■chow_alab_rep3
SRR1173469■hfd_wt_rep1
SRR1173470■hfd_wt_rep2
SRR1173471■hfd_wt_rep3
SRR1173472■hfd_j1c_rep1
SRR1173473■hfd_j1c_rep2
SRR1173474■hfd_j1c_rep3
SRR1173475■hfd_j2c_rep1
SRR1173476■hfd_j2c_rep2
SRR1173477■hfd_j2c_rep3
SRR1173478■hfd_alab_rep1
SRR1173479■hfd_alab_rep2
SRR1173480■hfd_alab_rep3
TSV
```

Create the nf-core samplesheet. Use `unstranded` for this A1 workflow to avoid the extra strandedness inference branch that requires additional tools:

```
awk 'BEGIN{print "sample,fastq_1,fastq_2,strandedness"} {print $2",/config/GSE55190/fastq/"$1"_1.fastq.gz,/c
onfig/GSE55190/fastq/"$1"_2.fastq.gz,unstranded"}' \
  runs_to_samples.tsv > GSE55190-benchmark-unstranded.csv
```

Download FASTQs from ENA:

```
curl -fL -o ena_fastq_urls.tsv \
'https://www.ebi.ac.uk/ena/portal/api/filereport?accession=SRP038635&result=read_run&fields=run_accession,fa
stq_ftp,fastq_md5&format=tsv'

tail -n +2 ena_fastq_urls.tsv | while IFS=$'\t' read -r run urls md5s; do
  IFS=';' read -r r1 r2 <<< "$urls"
  curl -fL --retry 5 -C - -o "fastq/${run}_1.fastq.gz" "https://${r1}"
  curl -fL --retry 5 -C - -o "fastq/${run}_2.fastq.gz" "https://${r2}"
done
```

Optional checksum check:

```
tail -n +2 ena_fastq_urls.tsv | while IFS=$'\t' read -r run urls md5s; do
  IFS=';' read -r m1 m2 <<< "$md5s"
  echo "${m1}  fastq/${run}_1.fastq.gz"
  echo "${m2}  fastq/${run}_2.fastq.gz"
done > fastq_md5s.txt

md5sum -c fastq_md5s.txt
```

Download Reference Files

The benchmark used mouse mm39 GENCODE M36 files:

```
cd /config/GSE55190

curl -fL -o reference/genome.fa \
  https://web.dolphinnext.com/umw_biocore/dnext_data/genome_data/mouse/mm39/gencode_m36/main/genome.fa

curl -fL -o reference/genes.gtf \
  https://web.dolphinnext.com/umw_biocore/dnext_data/genome_data/mouse/mm39/gencode_m36/genes/genes.gtf
```

Verify:

```
ls -lh reference/genome.fa reference/genes.gtf
grep -m 1 '^>' reference/genome.fa
head -3 reference/genes.gtf
```

Storage Recommendations

For production, avoid keeping large datasets only on the login node boot volume.

Preferred: OCI File Storage Service

Use FSS when the pipeline needs POSIX semantics, shared random reads, and Nextflow work directories visible to every Slurm node.

Recommended layout:

```
/config/GSE55190/reference
/config/GSE55190/fastq
/config/GSE55190/work-apptainer-arm64
/config/GSE55190/results
/config/apptainer-cache
/config/nextflow-home
```

FSS is the simplest storage model for Slurm and Apptainer because every node can read and write the same path.

OCI Object Storage

Object Storage is good for durable source data and final outputs, but less ideal for active Nextflow work directories. A practical model is:

1. Keep raw FASTQs and references in Object Storage.
2. Copy or sync them to FSS before the run.
3. Run Nextflow with FSS paths.
4. Sync final results back to Object Storage.

Example:

```
oci os object bulk-download \
  --bucket-name rnaseq-inputs \
  --download-dir /config/GSE55190/fastq

oci os object bulk-upload \
  --bucket-name rnaseq-results \
  --src-dir /config/GSE55190/results/full-apptainer-arm64
```

For very large production workloads, consider a higher-performance shared filesystem if FSS throughput becomes the bottleneck.

Install Apptainer on A1 Nodes

Install Apptainer on the login node and every compute node:

```
for n in wise-insect-login $(sinfo -N -h -o '%N' | sort -u); do
  echo "================ installing apptainer on $n ================"
  ssh -o BatchMode=yes "$n" 'sudo bash -s' <<'EOF'
set -euxo pipefail
apt-get update
apt-get install -y software-properties-common
add-apt-repository -y ppa:apptainer/ppa
apt-get update
apt-get install -y apptainer
apptainer --version
uname -m
EOF
done
```

Test Slurm plus Apptainer:

```
cat > /config/GSE55190/apptainer-slurm-test.nf <<'EOF'
process hello {
  container 'docker://arm64v8/ubuntu:22.04'
```

```
  output:
  stdout

  script:
  """
  hostname
  uname -m
  echo apptainer-slurm-ok
  """
}

workflow {
  hello.view()
}
EOF
```

Run:

```
NXF_APPTAINER_CACHEDIR=/config/apptainer-cache \
nextflow run /config/GSE55190/apptainer-slurm-test.nf \
  -c /config/GSE55190/nextflow-apptainer-arm64.config \
  -w /config/GSE55190/apptainer-test-work
```

Expected output includes:

```
aarch64
apptainer-slurm-ok
```

Build the ARM64 Toolbox Image

Official nf-core/Biocontainers images often target x86_64. On A1, use a custom ARM64 image with the required tools.

Create the definition:

```
cat > /config/GSE55190/containers/rnaseq-arm64-toolbox.def <<'EOF'
Bootstrap: docker
From: arm64v8/ubuntu:22.04

%post
    export DEBIAN_FRONTEND=noninteractive
    apt-get update
    apt-get install -y --no-install-recommends software-properties-common ca-certificates curl wget gnupg
    add-apt-repository -y universe
    apt-get update
    apt-get install -y --no-install-recommends \
        bash coreutils findutils grep sed gawk gzip bzip2 xz-utils tar unzip zip pigz procps \
        default-jre-headless \
        python3 python3-pip python-is-python3 \
        samtools bedtools bedops gffread kallisto fastqc cutadapt trim-galore \
        rsem rna-star \
        r-base r-base-dev \
        r-cran-data.table r-cran-jsonlite r-cran-optparse r-cran-readr r-cran-tibble r-cran-dplyr \
        r-bioc-tximport r-bioc-tximeta r-bioc-summarizedexperiment r-bioc-rhdf5 r-bioc-genomicranges r-bioc-
deseq2
    pip3 install --no-cache-dir multiqc pyyaml pandas

    cat > /usr/local/bin/gtf2bed <<'EOS'
#!/usr/bin/env bash
set -euo pipefail
if [[ $# -eq 0 ]]; then
  exec convert2bed --input=gtf
elif [[ $# -eq 1 && -f "$1" ]]; then
  exec convert2bed --input=gtf < "$1"
else
  exec convert2bed --input=gtf "$@"
fi
EOS
    chmod 0755 /usr/local/bin/gtf2bed

    apt-get clean
    rm -rf /var/lib/apt/lists/*

%environment
    export LC_ALL=C.UTF-8
    export LANG=C.UTF-8
    export PATH=/usr/local/bin:/usr/bin:/bin:$PATH
EOF
```

Build:

```
sudo apptainer build --force /config/GSE55190/containers/rnaseq-arm64-toolbox.sif \
  /config/GSE55190/containers/rnaseq-arm64-toolbox.def
```

Verify:

```
apptainer exec /config/GSE55190/containers/rnaseq-arm64-toolbox.sif bash -lc '
uname -m
python --version
samtools --version | head -1
gffread --version
kallisto version
trim_galore --version
fastqc --version
cutadapt --version
multiqc --version
Rscript -e "library(tximport); library(tximeta); library(DESeq2); cat(\"R packages OK\n\")"
which gtf2bed
'
```

Nextflow Configuration

Create `/config/GSE55190/nextflow-apptainer-arm64.config`:

```
process {
  executor = 'slurm'
  queue = 'compute'
  cpus = 1
  memory = '14.GB'

  withName: '.*' {
    arch = 'linux/arm64'
    container = 'file:///config/GSE55190/containers/rnaseq-arm64-toolbox.sif'
  }
}

executor {
  queueSize = 2
  submitRateLimit = '2 sec'
}

apptainer {
  enabled = true
  autoMounts = true
  cacheDir = '/config/apptainer-cache'
  pullTimeout = '60 min'
}

singularity.enabled = false
docker.enabled = false
conda.enabled = false
wave.enabled = false
```

For a larger A1 cluster with 50 single-OCPU nodes, set `queueSize = 50`. Keep `cpus = 1` unless each Slurm node has more CPUs.

Patch nf-core Software Version Metadata

The ARM64 toolbox can emit version strings that make nf-core/rnaseq `3.14.0` software-version collation fail. Patch the cached template before running:

```
export NXF_HOME=/config/nextflow-home
export NXF_SYNTAX_PARSER=v1
nextflow pull nf-core/rnaseq -r 3.14.0

python3 - <<'PY'
from pathlib import Path
for p in Path("/config/nextflow-home/assets/.repos/nf-core/rnaseq/clones").glob(
    "*/modules/nf-core/custom/dumpsoftwareversions/templates/dumpsoftwareversions.py"
):
    s = p.read_text()
    s = s.replace("import yaml\n", "import yaml\nimport json\n")
    s = s.replace(
        'with open("collated_versions.yml") as f:\n        versions_by_process = yaml.load(f, Loader=yaml.Ba
seLoader) | versions_this_module',
        '''raw = open("collated_versions.yml", "rb").read().decode("utf-8", "replace")
    safe_lines = []
    for line in raw.splitlines():
        if line.startswith("    ") and ": " in line:
            k, v = line.split(": ", 1)
            safe_lines.append(f"{k}: {json.dumps(v)}")
        else:
            safe_lines.append(line)
```

```
    versions_by_process = yaml.load("\\n".join(safe_lines), Loader=yaml.BaseLoader) | versions_this_module''
'
    )
    p.write_text(s)
    print("patched", p)
PY
```

Smoke Test

Create a 1-sample samplesheet:

```
{ head -n 1 GSE55190-benchmark-unstranded.csv; grep '^chow_wt_rep1,' GSE55190-benchmark-unstranded.csv; } \
  > GSE55190-test1-unstranded.csv
```

Run:

```
cd /config/GSE55190

export NXF_HOME=/config/nextflow-home
export NXF_SYNTAX_PARSER=v1
export NXF_APPTAINER_CACHEDIR=/config/apptainer-cache
export APPTAINER_CACHEDIR=/config/apptainer-cache
export APPTAINER_TMPDIR=/tmp

nextflow run nf-core/rnaseq -r 3.14.0 \
  -c /config/GSE55190/nextflow-apptainer-arm64.config \
  -w /config/GSE55190/work-apptainer-arm64 \
  --input /config/GSE55190/GSE55190-test1-unstranded.csv \
  --fasta /config/GSE55190/reference/genome.fa \
  --gtf /config/GSE55190/reference/genes.gtf \
  --outdir /config/GSE55190/results/test1-apptainer-arm64 \
  --pseudo_aligner kallisto \
  --skip_alignment \
  --skip_deseq2_qc \
  --gencode \
  --igenomes_ignore \
  --genome null \
  --save_reference \
  --max_cpus 1 \
  --max_memory '14.GB' \
  -with-report /config/GSE55190/results/test1-apptainer-arm64/report.html
```

Fresh Full Run

Clean only runtime artifacts:

```
cd /config/GSE55190
rm -rf /config/GSE55190/work-apptainer-arm64
rm -rf /config/GSE55190/results/full-apptainer-arm64
rm -f /config/GSE55190/.nextflow.log*
```

Run all 24 samples:

```
cd /config/GSE55190

export NXF_HOME=/config/nextflow-home
export NXF_SYNTAX_PARSER=v1
export NXF_APPTAINER_CACHEDIR=/config/apptainer-cache
export APPTAINER_CACHEDIR=/config/apptainer-cache
export APPTAINER_TMPDIR=/tmp

start_epoch=$(date +%s)
echo "START: $(date)"

nextflow run nf-core/rnaseq -r 3.14.0 \
  -c /config/GSE55190/nextflow-apptainer-arm64.config \
  -w /config/GSE55190/work-apptainer-arm64 \
  --input /config/GSE55190/GSE55190-benchmark-unstranded.csv \
  --fasta /config/GSE55190/reference/genome.fa \
  --gtf /config/GSE55190/reference/genes.gtf \
  --outdir /config/GSE55190/results/full-apptainer-arm64 \
  --pseudo_aligner kallisto \
  --skip_alignment \
  --skip_deseq2_qc \
  --gencode \
  --igenomes_ignore \
  --genome null \
  --save_reference \
  --max_cpus 1 \
  --max_memory '14.GB' \
  -with-report /config/GSE55190/results/full-apptainer-arm64/report.html
```

```
status=$?
end_epoch=$(date +%s)
echo "END: $(date)"
echo "EXIT_STATUS: $status"
echo "ELAPSED_SECONDS: $((end_epoch - start_epoch))"
echo "ELAPSED_MINUTES: $(((end_epoch - start_epoch + 59) / 60))"
```

Monitor:

```
watch -n 30 'squeue -u ubuntu -o "%.18i %.9P %.45j %.8u %.2t %.12M %.6D %R %N"; echo; sinfo -N -p compute'
```

Output Checks

After completion:

```
find /config/GSE55190/results/full-apptainer-arm64 -maxdepth 3 -type f | sort | head -100
```

Important outputs should include:

- Kallisto quantification directories
- `tximport` matrices
- summarized gene and transcript count outputs
- MultiQC output, unless MultiQC/report rendering fails

If Nextflow says the pipeline completed successfully but report rendering warns, treat the analysis outputs as valid and inspect `.nextflow.log` separately for the report-rendering warning.